Unit-05
"Exception Handling"

Topic List :

- Errors and Exception.
- Try-Catch, Multiple-Catch and Finally blocks.
- Throw, Throws, Finally Keywords .
- Exception Handling and Best Practices.

Topic –01 "Errors and Exception"

- **Errors :**
- Errors are serious problems that occur due to system-level failures and are generally beyond the control of the application. They are not meant to be caught or handled by the program.
- They indicate that something has gone wrong with the environment running your Java application. You should **not** attempt to catch Errors in code.



Example:

- **Out of memory error -**

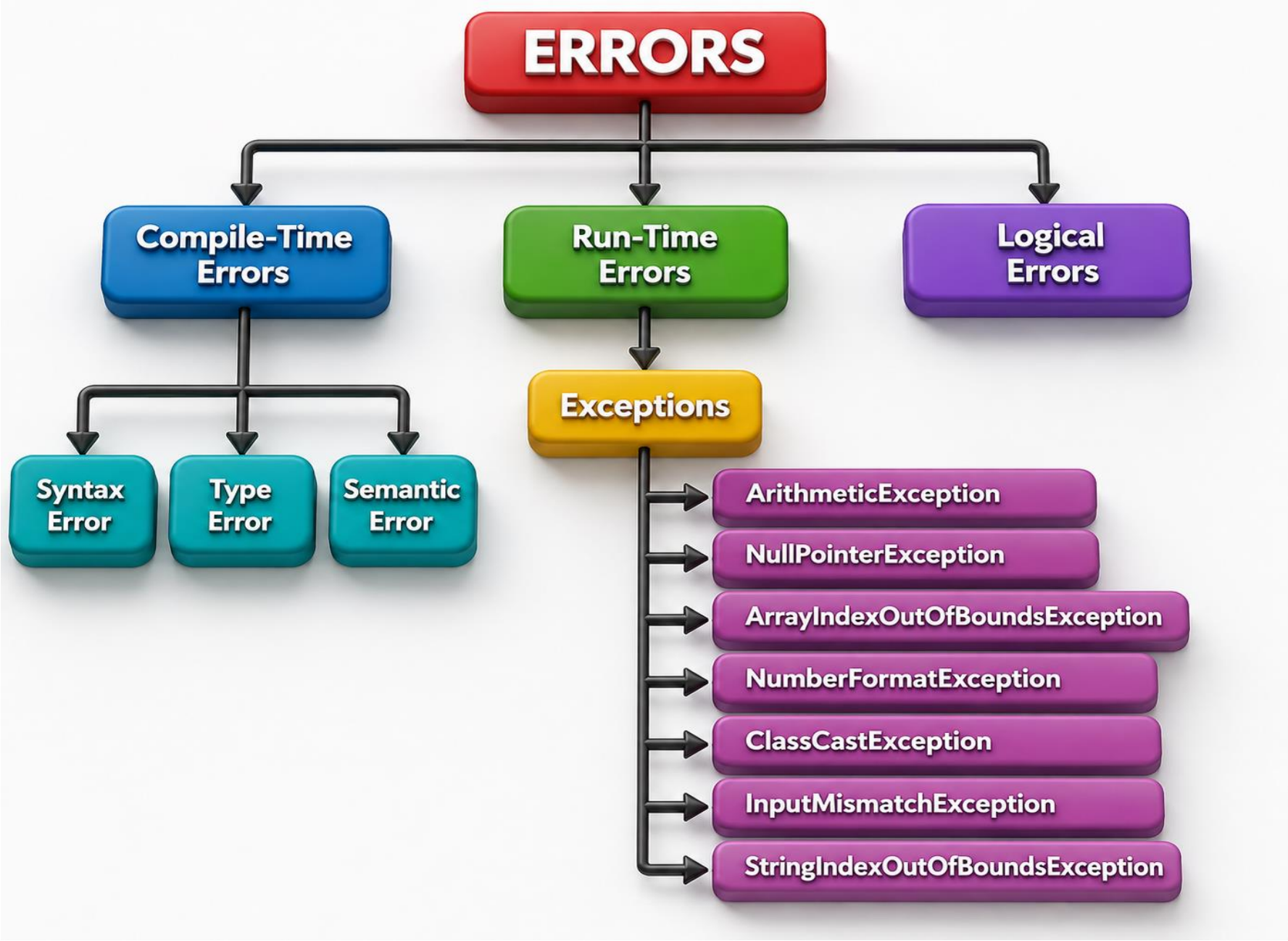
```
Public class outofmemory
{
    Public static void main(String[] args)
     {
        Int arr[] = new int(Integer.MAX_VALUE);
     }
}
```

- **Stack overflow error -**

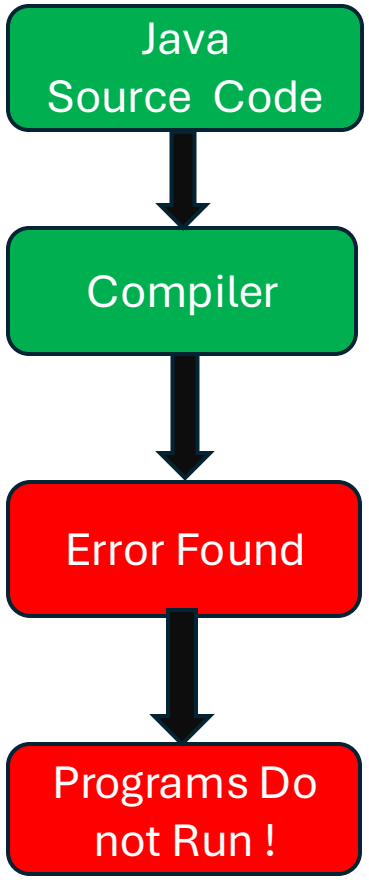
```
 Public class Stackoverflow
 {
     Public class recursiveMethod()
      {
         RecursiveMethod();
}
```

```
 Public static void main(String[] args)
 {
     RecursiveMethod();
 }
}
```

"Types of Errors "



- **Compile- time Error :**
  - A Compile- time error occurs when the java compiler finds a problem before the programs run .
  - The Compiler Checks your .java code and converts it into bytecode.



Sub-types of Compile-time Error :

➢ **Syntax Error :** These Occurs when We don't follow Java syntax /rules .

Example 1 – "Missing Semicolon"

```
Public class Error {
Public static void main(String[] args) {
int a =10
} }
```

Example 2 – "Incorrect Brackets"

```
Public class Error {
Public static void main(String[] args) {
If(a>10 {
S.O.P(a);
}
```

Example 3 – "Incorrect Spelling of keyword"

```
Publc class main
```

- **Type Error :** These type of error occur when incompatible data types are used.

Example :

```
Public class Errors {
 Public static void main(String[] args) {
  int a = "Hello" ;
  }}
```

- **Semantic Error :** The Syntax may look Correct, but Statement violates java rules .

Example :

```
Public class main {
  Public static void main(String[] args) {
    int a =10;
    S.O.P(b);
 }}
```

- **Undeclared variable Error :** This type of Error when we try to use a variable that hasn't been declared.

Example :

```
Public class main {
  Public static void main(String[] args) {
  System.out.println("The Age will be : " +age);
  } }
```

- **Method / argument Error :** This type of Error occur when the method call doesn't match the method definition .

Example :

```
Public class main {
    Public void addition(int a , int b) {
      S.O.P("Addition : " +(a+b));
    Public static void main (String[] args) {
       addition(10,20,30);
    } }
```

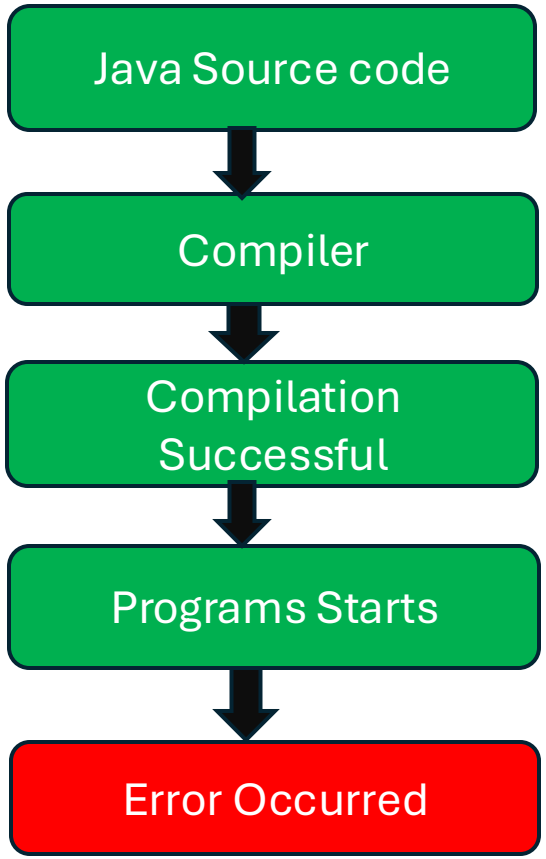
- **Class name / file name error :** This type of error occur when the file name and class name doesn't match.

  Example : file name : Test.java

  ```
  Public class main {

  }
  ```

- **Run- time Error :**
  - A run-time error occur after the program has successfully compiled and started running
  - Most java run-time error appear as Exception.



Sub-types of Run – Time Error :

- **Arithmetic Exception -** This Type of Exception occurs when an invalid arithmetic operation performed. Most common divisible by 0.

  Example :

```
Public class main {
  Public static void main(String[] args) {
    Int a = 10;
    Int b =0;
    S.O.P(a/b);
}}
```

- **ArrayIndexOutOfBounds Exception -** This Type of Exception occurs when we try to access an array index that doesn't access.

  Example :

```
Int[] arr = {10,20,30,40};
S.O.P("value at index no 5 will be : " +arr[5]);
```

- **Null Pointer Exception** – This type of Exception occur when we try to use a object / reference that contains NULL.

  Example :

  String name = null;

  S.O.P(name.length());

- **NumberFormatException –** This type of Exception occur when we try to convert an invalid string into number.

  Example:

  String S ="Hello";

  Int num = Integer.parseInt(s);

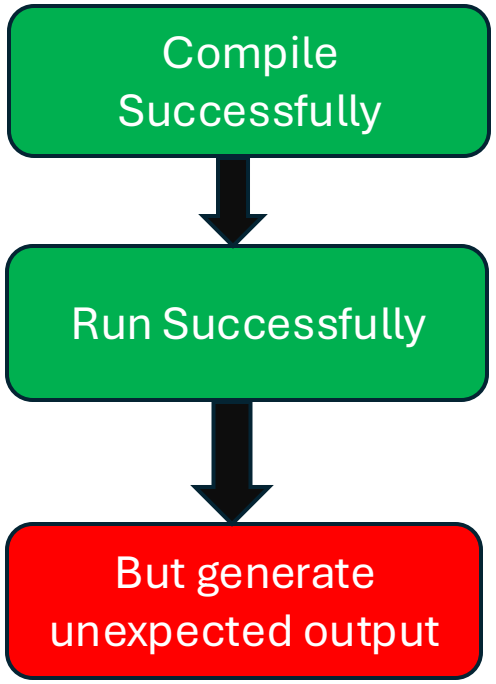
- **Class Cast Exception –** This type of Exception occur when an object is incorrectly cast to an incompatible type .

  Example:

  Object obj = "Hello";

  Integer num = (Integer)obj;

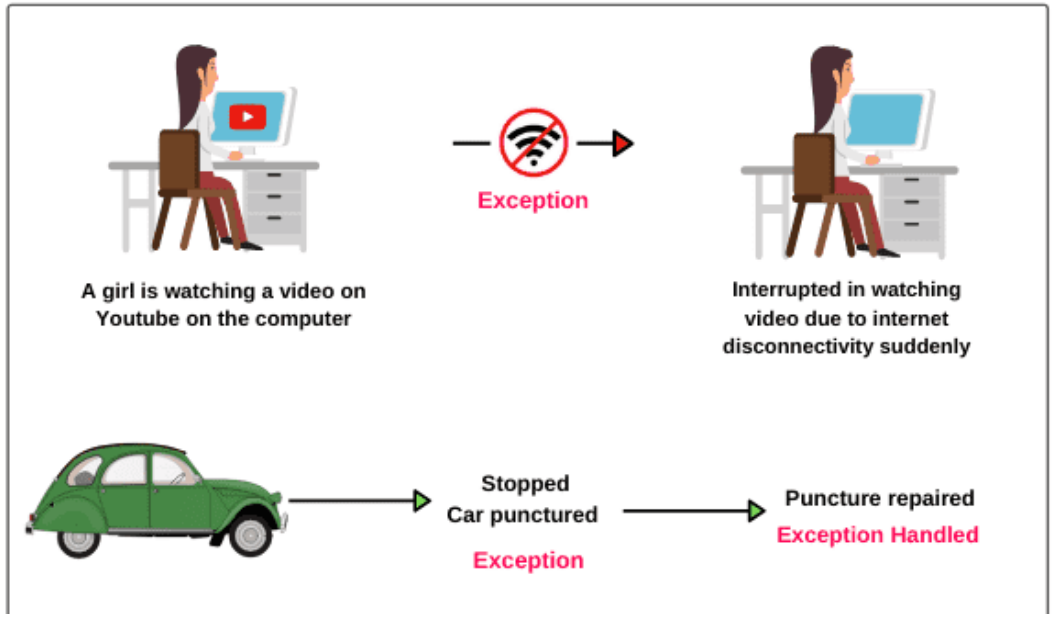
- **Logical Errors :**
  - In Logical Error, the logic is incorrect or invalid .
  - We want addition as an output, but we get subtraction in output .



Example :

```
Public class addition {
  Public static void main(String[] args) {
    int a=10;
    int b=20;
    System.out.println("addition of two number : " +(a-b));
  }}
```

"Exception"



- An exception in java is an unexcepted event or problem that occurs while a program is running and interrupts the normal flow of the program.
- It can be handled through Exception Handling.
- Helps maintains program stability by preventing unexpected termination.

EXCEPTIONS IN JAVA

1. BUILT-IN / PREDEFINED EXCEPTIONS (Provided by Java)

A. CHECKED EXCEPTIONS (Checked at Compile Time)

Compiler checks these exceptions. Must be handled or declared using throws.

1. **IOException**
   Input/Output operation failed.
   - 2. **FileNotFoundException**
     File cannot be found or accessed.
     (Child of IOException)
3. **ClassNotFoundException**
   Required class cannot be found.
4. **NoSuchMethodException**
   Required method does not exist.

B. UNCHECKED EXCEPTIONS (Checked at Runtime)

Occurs at runtime. Compiler does not force us to handle these.

1. **ArithmeticException**
   Invalid arithmetic operation (e.g., divide by zero).
2. **NullPointerException**
   Using a null reference.
3. **StringIndexOutOfBoundsException**
   Invalid index used to access a String.
4. **NumberFormatException**
   Invalid conversion of String to number.
5. **ArrayIndexOutOfBoundsException**
   Invalid index used to access an array.
6. **IllegalArgumentException**
   Invalid or inappropriate argument passed to a method.
7. **IllegalStateException**
   Method called at an inappropriate time or in an invalid state.

2. USER-DEFINED / CUSTOM EXCEPTIONS (Created by Programmer)

Programmer creates his own exception classes according to application requirements.

EXAMPLE

```
class AgeException extends Exception
{
    AgeException(String message)
    {
       super(message);
    }
}

class Main
{
   public static void main(String[] args)
   {
      int age = 15;
      try
         if(age < 18)
            throw new AgeException("Age must
                        be 18 or above");
         System.out.println("Eligible");
      }
      catch(AgeException e)
      {
          System.out.println(e.getMessage());
      }
   }
}
```

OUTPUT

```
Age must be
18 or above
```

SUMMARY : Java Exceptions are of two types – Built-in (Checked & Unchecked) provided by Java and User-Defined (Custom) created by the programmer.

Built-in / pre-defined Exception:-

A) Checked Exception -

- The Compiler Checks these exception . We Normally use Try-catch block or declare them using throws.

I/OException : An I/O Exception Occurs when java program has a problem to reading data or writing data.

It belongs to java.io package .

Example -

```
import java.io.*;
public class Main
{
   public static void main(String[] args) throws IOException
   {
      FileReader file = new FileReader("data.txt");
      int ch = file.read();
      System.out.println((char) ch);
      file.close();
   }
}
```

ClassNotFoundException : ClassNotFoundException occurs when java tries to load class at runtime, but it cannot find that class. It belongs to java.lang package.

Example -

```
public class Main
{
   public static void main(String[] args) throws ClassNotFoundException
   {
      Class.forName("Student");
      System.out.println("Class Found");
   }
}
```

NoSuchMethodException : NoSuchMethodException occurs when java tries to load but it cannot find that method. It is also belongs to java.lang package.

Example -

```
class student
{
  public void display()
  {
     System.out.println("Student details");
  }
}
public class Main
{
    public static void main(String[] args) throws NoSuchMethodException
    {
       student.class.getMethod("show");
    }
}
```

B)Unchecked Exception -

- An Unchecked Exception is an exception that java does not force programmer to handle at compile time.
- It usually occurs because of programming mistake or invalid input during runtime.

ArithmeticException : ArithmeticException Occurs when java encounters invalid arithmetic operation during program execution .

Example -

```
public class Main
{
   public static void main(String[] args)
   {
   int a=10;
   int b=0;
   System.out.println("division: "+a/b);
   }
}
```

NullPointerException : NullPointerException occurs when we try to use an object reference that contains Null.

Example -

```
public class Main
{
   public static void main(String[] args)
   {
      String name = null;
      System.out.println(name.length());
   }
}
```

StringIndexOutOfBoundException: StringIndexOutofBoundException occurs when we try to access a character at invalid index of a string.

Example -

```
public class Main
{
   public static void main(String[] args)
   {
     String name ="Java";
     System.out.println(name.charAt(5));
   }
}
```

NumberFormatException: NumberFormatException occurs when we try convert a string into a number , but the string does not contains valid integer .

Example -

```
public class Main
{
   public static void main(String[] args)
   {
      String name ="Java";
      int number = Integer.parseInt(name);
      System.out.println(number);
   }
}
```

ArrayIndexOutOfBoundException : ArrayIndexOutOfBoundException Occurs when we try access element from array using Invalid Index Number.

Example -

```
public class Main
{
   public static void main(String[] args)
   {
      int[] array = {1,2,3,4};
      System.out.println("Element on Index no 05 will be : " +array[5]);
   }
}
```

IllegalArgumentException : IllegalArgumentException Occurs when we pass an invalid or inappropriate argument to the method.

Example -

```
class Main
{
   static void setAge(int age)
   {
      if (age < 0)
       {
          throw new IllegalArgumentException("Age cannot be negative");
       }
       System.out.println("Age: " + age);
   }
   public static void main(String[] args)
   {
      setAge(-5);
   }
}
```

IllegalStateException : IllegalStateException Occurs when a method is called at a time when the object or program is in a inappropriate state for that operations.

Example -

```
class BankAccount
{
   boolean closed = true;

   void withdraw()
   {
      if (closed)
      {
         throw new IllegalStateException("Account is closed");
      }
      System.out.println("Money withdrawn");
   }
}
```

```
class Main {
  public static void main(String[] args) {

    BankAccount account = new BankAccount();

    account.withdraw();
  }
}
```

User – defined / Custom Exception :-

A user- defined / custom Exception , is an exception created by programmer for specific situation in program.

Java Already have exceptions such as Arithmetic Exception , Null pointer Exception but sometimes we need our own Exception.

Example -

If a person Age is less than 18 we want to show " Not Eligible for vote " . We want to create our own exception called InvalidAgeException.

```
class InvalidAgeException extends Exception
{
   InvalidAgeException(String message)
   {
      super(message);
   }
}
```

```
class Main{
   public static void main(String[] args)
   {
      int age = 15;
      try{
         if(age < 18){
            throw new InvalidAgeException("Age must be 18 or above");
         }
         System.out.println("Eligible for voting");
      }
      catch(InvalidAgeException e)
      {
         System.out.println(e.getMessage());
      }
   }
}
```

Topic :-02
"Try-catch , multiple catch , finally block"

Try-catch Block :

Try-catch block is generally used to handle exceptions in java, so program does not stop suddenly when an errors occurs .

try - "Try to Execute this code".

catch - "If an Exception occurs handle, it here".

Syntax -

```
try { // code that may cause an exception.

}

catch(Exception e) {// code to handle the exception.

}
```

Example -

```
public class Main
{
   public static void main(String[] args)
   {
      try {
         int a=10;
         int b=0;
         System.out.println("Divison of two numbers will be: " +(a/b));
      }
      catch(Exception e)
      {
         System.out.println("Cannot divide by zero");
      }
   }
}
```

Multiple-catch block :

A multiple-catch block means using more than one catch block with a single try block .

It can be used when a single try block generates multiple Exception and we want to handle each exception differently.

Syntax :

```
Try {
}
Catch(Exception e) {
}
Catch(Exception e) {
}
```

Example -

```
public class Main {
   public static void main(String[] args) {
      try {
         String name=null;
         System.out.println("lenth of name will be :"+name.length());
         int a=10;
         int b=0;
         System.out.println("Divison of two numbers will be: " +(a/b));
      }
      catch(NullPointerException e) {
         System.out.println("Value cannot be null");
      }
      catch(ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      }
   }
}
```

Finally block :

Finally block is used to write that code we want to execute whether an exception occur or not .

Try - "try to execute this code".

Catch -"Handle the error"

Finally - "Do this at the end".

Syntax :

```
Try {
}
Catch(Exception E){
}
Finally{
}
```

Example :

```
public class Main {
   public static void main(String[] args) {
      try {
         String name=null;
         System.out.println("lenth of name will be :"+name.length());
         int a=10;
         int b=0;
         System.out.println("Divison of two numbers will be: " +(a/b));
      }
      catch(NullPointerException e) {
         System.out.println("Value cannot be null");
      }
      catch(ArithmeticException e) {
         System.out.println("Cannot divide by zero");
      }
      finally {
         System.out.println("Finally block executed");
      }
  }
}
```

Topic:-03
"Throw, Throws, Finally keywords"

Throw Keyword :

- The throw keyword in java is used to explicitly throw an exception from a method or any block of code .
- We can throw wither checked or unchecked Exception .
- Throw keyword in mainly used to throw custom Exception.

Syntax :

Throw new exception name ("message");

Example:

```
public class Main
{
   public static void main(String[] args)
   {
      int age=15;
      if(age<18)
      {
         throw new ArithmeticException("Not Eligible");
      }
      System.out.println("Eligible");
   }
}
```

Throws keyword :

- Throws keywords used in signature of methods to indicate that this method might throw one of the listed type exception.
- Throws is a declaration and act as a warning system or indication for anyone calling a particular method.
- It is mainly used with checked Exception.

Syntax :

Returntype MethodName throws ExceptionName

Example -

```
import java.io.*;
public class Main
{
   public static void readfile() throws IOException
   {
      FileReader f = new FileReader("data.txt");
   }
```

```
public static void main(String[] args)
   {
     try{
        readfile();
     }
     catch(IOException e)
     {
        System.out.println("File not found !! ");
     }
   }
}
```

Finally keywords :

The Finally block is used alongside try and catch to define a piece of code that always executes , regardless of whether exception occurs or was caught.

Syntax :

```
try{
}
catch(Exception e) {
}
finally {
}
```

Example -

```
public class Main
{
   public static void main(String[] args)
   {
```

```
try
    {
      int a = 10;
      Int b = 0;
      System.out.pritnln("Division of two number will be: "+(a/b));
    }
    catch(ArithmeticException e)
    {
      System.out.println("Exception handled");
    }
    finally
    {
      System.out.println("Finally block executed");
    }
  }
}
```

Topic –04
"Exception Handling and Best practices"

" Exception handling "

- ✓ Exception handling is the mechanism in java used to handle runtime problems / error without terminating the program.
- ✓ Exception Handling allows normal flow of program by handling problem . without Exception Handling program terminated at that point.
- ✓ Exception Handling mainly perform through try and catch blocks.

Syntax -

```
Try {
}
Catch(Exception e) {
}
```

Example -

```
class Exceptionhandling
{
   public static void main(String[] args)
      {
         try {
         int a=10;
         int b=0;
         System.out.println("Division of two number will be: " +(a/b));
         }
         catch(ArithmeticException e)
         {
            System.out.println("Cannot divide by zero ");
         }
      }
}
```

" Main Keywords of Exception Handling "

- Try : Contains that part of code which may generate Exception.
- Catch: It is used to handle exception.
- Finally: used for code that should execute after try/catch , especially cleanup.
- Throw: The throw keyword is used to manually create and throw an exception in java.
- Throws: The throws keyword is used to tell the caller of a method that " this method may generate an exception so you must handle it ."

"Best Practices "

Good Exception handling is very important in real – world java applications.

- **Catch Specific Exceptions :**

```
Catch(Exception e)
{
System.out.println("Error");
}
```

When you know the specific exception

```
Catch(ArithmeticException e)
{
System.out.println("Cannot divide by zero");
}
```

Because specific Exceptions make program easier to understand and debug.

- **Don't use empty Catch blocks :**

```
Catch(exception e)
{
}
```

This hides the problem.

```
Catch(exception e)
{
System.out.println("Error : " +e.getmessage());
}
```

- **Give meaningful message:**

```
System.out.println("Error");
```

A meaningful message helps programmer and user understand what happened.

```
System.out.println("File could not be found ");
```

- **Don't expose sensitive information :**

Avoid showing technical information such as database passwords, API keys, or internal system details to users.

Instead of showing a complete technical error to the user.

Database Connection failed at server XYZ..

Show something simple -

Unable to process your request . Please try again later